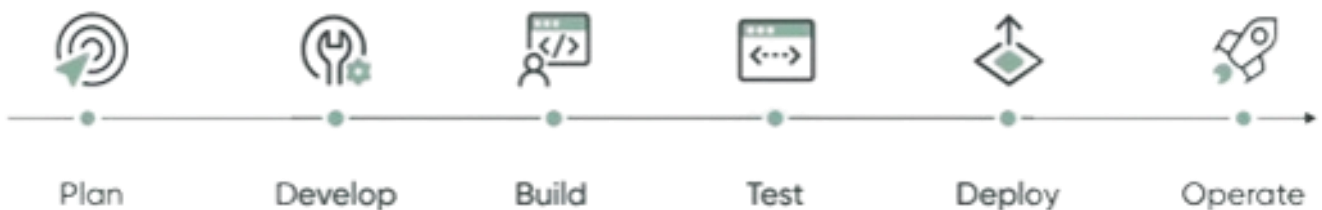




HOGESCHOOL ROTTERDAM / CMI

Software construction: process and tools



INFSCP22

Aantal studiepunten: 12
Cursushouder: S. de Redelijkheid



Cursusnaam:	Software construction: Process and tools																											
Cursuscode:	INFSCP22																											
Aantal studiepunten en studiebelastinguren:	Dit studieonderdeel levert de student 12 studiepunten op, hetgeen overeenkomt met een studielast van 336 uren. De verdeling van deze 336 uren over de collegeweken is als volgt: <table><tr><th colspan="3">Begeleide lessen</th></tr><tr><td>Kick-off bijwonen</td><td>1 x 2 uur</td><td>2 uur</td></tr><tr><td>Lessen inclusief workshops</td><td>16 weken x 4 uur</td><td>128 uur</td></tr><tr><td>Einddemonstratie geven en deelname</td><td>1x 4 uur</td><td>4 uur</td></tr><tr><th colspan="3">Onbegeleid</th></tr><tr><td>Werken aan het projectdeel</td><td>16 weken * 9 uur</td><td>144 uur</td></tr><tr><td>Zelfstudie en voorbereiding</td><td>16 weken * 7 uur</td><td>112 uur</td></tr><tr><td>Vorbereiding einddemonstratie</td><td>1 x 5 uur</td><td>5 uur</td></tr><tr><td colspan="2">Totaal</td><td>336 uur</td></tr></table>	Begeleide lessen			Kick-off bijwonen	1 x 2 uur	2 uur	Lessen inclusief workshops	16 weken x 4 uur	128 uur	Einddemonstratie geven en deelname	1x 4 uur	4 uur	Onbegeleid			Werken aan het projectdeel	16 weken * 9 uur	144 uur	Zelfstudie en voorbereiding	16 weken * 7 uur	112 uur	Vorbereiding einddemonstratie	1 x 5 uur	5 uur	Totaal		336 uur
Begeleide lessen																												
Kick-off bijwonen	1 x 2 uur	2 uur																										
Lessen inclusief workshops	16 weken x 4 uur	128 uur																										
Einddemonstratie geven en deelname	1x 4 uur	4 uur																										
Onbegeleid																												
Werken aan het projectdeel	16 weken * 9 uur	144 uur																										
Zelfstudie en voorbereiding	16 weken * 7 uur	112 uur																										
Vorbereiding einddemonstratie	1 x 5 uur	5 uur																										
Totaal		336 uur																										
Vereiste voorkennis:	Kennis van jaar 1 (Basecamp, OODP en Project)																											
Werkvorm:	Projectmatig onderwijs met workshops (teamwork)																											
Toetsing:	Examinering op basis van ingeleverd projectwerk en het proces.																											
Leermiddelen:	Geén verplichte literatuur																											
Draagt bij aan competentie:	Manage & Control, Analyseren, Ontwerpen, Adviseren en Realiseren																											
Leerdoelen:	1. De student kan bestaande software analyseren, documenteren en valideren 2. De student kan effectieve refactoring technieken toe passen om de code te verbeteren en uit te bereiden 3. De student kan samenwerken met Git als versiebeheersysteem. Er kunnen branches gemaakt, samengevoegd en beheerd worden 4. De student kan automatische tests schrijven en uitvoeren, waaronder unit tests en integratie tests 5. De student kan een CI/CD pipeline configureren binnen GitHub Actions of GitLab CI/CD met stappen voor build, testen en implementatie 6. De student kent de basisprincipes van containerization en is in staat deze toe te passen met behulp van Docker of Podman 7. De student is zich bewust van beveiligingsaspecten, zoals het veilig opslaan van credentials, encryptie en het controleren van kwetsbaarheden dependencies 8. De student kan effectief samenwerken in een teamomgeving, waarbij code wordt beoordeeld, feedback wordt gegeven en communicatie plaatsvindt																											
Inhoud:	Je leert werken in een projectgroep (maximaal 4 personen) om bestaande software te analyseren, documenteren en testen. Samen met je projectgroep ga je de bestaande software aanpassen (refactoren) en deze koppelen aan automatiseringstools binnen CI/CD, om deze vervolgens uit te breiden aan de hand van wensen van de Product Owner (client).																											
Opmerkingen:	Aanwezigheid is verplicht.																											
Cursushouder:	S. de Redelijkheid																											
Datum:	7 juni 2026																											



1. Algemene omschrijving

“The most powerful tool we have as developers is automation.” — Scott Hanselman

1.1 Inleiding

Als software engineer ben je altijd op zoek naar manieren om dingen voor jou te laten werken. In deze cursus ga je in op automatiseringsslagen voor het ontwikkeltraject en de verbeteringen die je kan toepassen om een zo goed mogelijk stuk software af te leveren. Samen met je team ga je aan de slag met bestaande software, deze breng je in kaart, voorzie je van documentatie (al dan niet geautomatiseerd) en tests om te garanderen dat alles werkt zoals het zou moeten. Als je de basis hebt staan, ga je door middel van automatisering via CI/CD ervoor zorgen dat aanpassingen aan de software nog steeds werken. Als je samen met je team een werkende pipeline hebt opgezet, ga je beginnen met het uitbreiden of verbeteren van de software. Bij de lessen zal om de week de skills docent aanwezig zijn.

1.2 Relatie met andere onderwijseenheden

In het eerste jaar heb je leren programmeren in Python en C# waarbij verschillende concepten en moeilijkheidsgraden aan bod zijn gekomen. Dit jaar gaan we verder op de kennis die je hebt opgebouwd en deze uitbreiden met automatiseringsmogelijkheden die je helpen in je leertraject, stage en verdere carrière.

1.3 Leermiddelen

Learning GitHub Actions van O'Reilly Media er op komen [optioneel]
(<https://www.oreilly.com/library/view/learning-github-actions/9781098131067/>)

Learning UML 2.0 [optioneel]
(<https://www.oreilly.com/library/view/learning-uml-20/0596009828/>)

Efficient Linux at the Command Line [optioneel]
(<https://www.oreilly.com/library/view/efficient-linux-at/9781098113391/>)

Refactoring: Improving the Design of Existing Code [optioneel]
(<https://www.oreilly.com/library/view/refactoring-improving-the/9780134757681/>)

Effective Software Testing [optioneel]
(<https://www.oreilly.com/library/view/effective-software-testing/9781633439931/>)

Python Testing with pytest [optioneel]
(<https://www.oreilly.com/library/view/python-testing-with/9781680509427/>)



2. Programma

Je komt als ontwikkelaar te werken in een klein team (max 4 studenten), waarbij je samen aan bestaande (legacy) software in Python gaat werken op basis van de beschikbare casus. Gaandeweg dien je de software te analyseren, in kaart te brengen, te testen en vervolgens te refactoren. Tegelijkertijd ga je een CI/CD pipeline opzetten om gedurende het ontwikkeltraject altijd te blijven testen, loggen en monitoren. Als alles goed gaat, breidt je de applicatie uit met extra functionaliteit.

2.1 Technische requirements

Je begint het project in Python maar het staat je vrij om later te wisselen naar andere talen/frameworks/tools voor de uitwerking en verbetering van de applicatie. Zorg ervoor dat je dit bespreekt met je technisch docent om te zien of jouw keuze haalbaar is. Het is belangrijk om met de gekozen technologieën aan de criteria uit rubric (3.2) te voldoen. Als student heb je de mogelijkheid om zowel GitHub als GitLab te gebruiken, voor GitHub kan je als student gebruikmaken van het GitHub Student Developer Pack (<https://github.com/edu/students>), welke je vele extra's geeft en de mogelijkheid biedt om als Team te kunnen werken. Voor GitLab kan je de free version gebruiken of de GitLab server van de HRO gebruiken (vraag je technisch docent).

Voor beide oplossingen kan het voorkomen dat de hoeveelheid minuten voor de runners niet toereikend is, dan kan de school je voorzien van een Virtual Machine (VM) server die je kunt aanvragen via het volgende formulier: <https://forms.office.com/e/Nc0DXbXTUy> of bij de receptie op de begane grond van Wijnhaven 99, vlakbij de ingang. Hierop kan je dan je private runner installeren en koppelen om te gebruiken binnen GitHub Actions of GitLab.

2.2 Opleveren

Aan het einde van de cursus lever je een deel van het werk in als team en een deel op individuele basis, hieronder is opgesplitst wat ingeleverd dient te worden:

Team

- Documentatie toegepaste projectmethodiek (scrum)
- Werkende CI/CD pipeline
 - o Video werkende pipeline (max. 720p, max. 2 minuten)
 - o YAML bestanden
 - o Bash bestanden
 - o Configuratie en dependency bestanden
- Docker file(s) en gerelateerde bestanden
- Aangepaste applicatie (code)
- Documentatie
 - o Onderzoeksrapport
 - o Technisch rapport
 - o Testrapport
 - o Swagger documentatie
- Presentaties
 1. Analyse bestaande software
 2. Testing
 3. Eindpresentatie

Individueel

- Schriftelijk bewijs van de individuele bijdrage aan het proces (een logboek met taken en uren) Maak duidelijk dat je aan de voorwaarden voldoet (zie 3.1).
- Schriftelijk bewijs van de individuele bijdrage aan het product, de pipeline en de rapporten (codefragmenten, samenvatting van bijdrage/voltooiden taken, git-logs). Let op: plagiaat wordt niet getolereerd;



2.3 Planning

Week	Uitvoering	Onderwerpen	Taken
1	Kick-off	Groepen vormen, casussen verdelen, opzetten omgevingen en samenwerkingscontract opstellen	
2	Workshop 1	REST, Web-API's, Requirements specifications, reverse-engineering	
3	Workshop 2	Opzetten van Git, aanmaken van branches en starten met het project op basis van issues.	
4	Workshop 3	Voordelen van testen, unit-, API-, integratie-, beveiliging & autorisatie-, prestatietesten	
5	Workshop 4	Het herkennen van security vulnerabilities bij een API	
6	Workshop 5	Refactoren van de codebase en voortgang bespreken	Inleveren concept onderzoeksrapport
7		Refactoren van de codebase en voortgang bespreken	Presentatie 1
8	Workshop 6	Voordelen van CI/CD, kiezen van een platform (GitHub/Gitlab), aanmaken van eerste pipeline	Bespreken feedback concept onderzoeksrapport
9		Het uitbreiden van de CI/CD pipelines en het opzetten van meerdere steps/jobs en voortgang bespreken	
10	Workshop 7	Werken met bash en taken automatiseren en voortgang bespreken	Inleveren concept testrapport
11	Workshop 8	Containerization, hoe zet je een container op en waarom gebruik je deze	Bespreken testrapport
12		Refactoren van de codebase (<i>database toevoegen</i>) en voortgang bespreken	Presentatie 2
13	Workshop 9	UML en ERD maken voor het technisch rapport en voortgang bespreken	
14		Werken aan het product en de documentatie en voortgang bespreken	
15		Werken aan het product en de documentatie en voortgang bespreken	Inleveren concept technisch rapport
16		Werken aan het product en de documentatie en voortgang bespreken	Bespreken technisch rapport
17		Werken aan het product en de documentatie en het inleveren van alle documentatie en bijdragen	Alles inleveren
18	Demo day	Het tonen van het eindresultaat in een demo	Presentatie 3
19		<i>Werken aan herkansing (indien nodig)</i>	
20	Resit final	<i>Werken aan herkansing (indien nodig)</i>	Herkansing inleveren



2.4 Workshops

Workshop 1 - Requirements Specification

In deze workshop gaan we kijken naar het opstellen van requirements. Wat zijn goede requirements en waar moeten ze o.a. aan voldoen. Wat is REST en hoe werken API's, een korte introductie en hoe je requirements daarvoor opstelt. Aan de hand van voorbeelden ga je kijken naar hoe je o.a. requirements uit bestaande software zou kunnen ontleiden.

Workshop 2 - Git

In deze workshop gaan we kijken naar het gebruik van Git en GitHub. Welke termen worden er gebruikt en waar dienen ze voor, wat is de normale gang van werken met Git en wat is het verschil met GitHub. Gedurende de gehele cursus is Git de levensader van CI/CD en nu kijken we naar waarvoor, waarom en hoe je het gebruikt.

Workshop 3 - Software Testing

In deze workshop gaan we kijken naar het testen van software. Wat test je wanneer, welke methoden zijn er om te testen en waar moet je rekening mee houden. Welke tools kan je gebruiken en hoe gaat dit straks helpen binnen onze CI/CD pipeline.

Workshop 4 - Security

In deze workshop gaan we kijken naar een aantal belangrijke security aspecten die relevant zijn bij het opzetten van een API. Hoe voorkom je dat kwaadwillende toegang krijgen tot gevoelige informatie en hoe minimaliseer je de schade als het ze toch lukt om binnen te komen?

Workshop 5 - Refactoring

In deze workshop gaan we kijken naar refactoring, wat is het, waar gebruik je het voor en hoe pas je het toe? Hoe draagt refactoring bij aan het beter onderhoudbaar maken van je software en hoe kunnen we dit op een bestaand project toepassen?

Workshop 6 - CI/CD

In deze workshop gaan we kijken naar Continuous Integration (CI) en Continuous Development (CD), wat wordt er mee bedoeld, waar gebruiken we het voor en wat is het verschil tussen Continuous Development en Continuous Delivery. We kijken naar tools, instellingen en gaan een begin maken met de opzet.

Workshop 7 - Operating Systems + Bash

In deze workshop gaan we kijken naar het gebruik van Bash en de kracht binnen CI/CD ervan. Hoe verschillende Operating Systems van elkaar verschillen. Hoe je navigeer je tussen mappen, hoe je maak je bestanden aan, hoe kan je bestanden verplaatsen en verwijderen en hoe stellen we rechten in.

Workshop 8 – Containerization

In deze workshop gaan we kijken naar Containerization, wat containers zijn en waar ze voor gebruikt worden. Hierbij kijken we naar het opzetten van simpele containers in Docker/Podman en hoe deze gebruikt kunnen worden om een einde te maken aan de kreet "Het werkt op mijn PC".

Workshop 9 – UML

In deze workshop gaan we dieper in op het gebruik en de kracht van UML als taal voor het visueel weergeven van software of veranderingen in software. Wij kijken onder andere naar de volgende onderdelen: Context diagram, ERD, Deployment diagram, Flowchart, State diagram, Swagger.



3. Toetsing en beoordeling

3.1 Beoordeling

Het op te leveren werk is een teamprestatie, waaraan iedereen individueel bijdraagt. Het cijfer wordt aan het team gegeven, maar er is een mogelijkheid om een verhoging (of verlaging) op individuele basis van maximaal +1 / -1 punt te krijgen op het groeps cijfer (zie 3.4). De beoordeling zal plaatsvinden door beide docenten (technisch en skills).

Individuele studenten die geen significante bijdrage leveren, kunnen echter worden uitgesloten van deze cursus en krijgen een No Go (een ND in Osiris). De docenten monitoren de bijdrage regelmatig en kunnen tijdens de cursus om verduidelijking vragen. Om een cijfer te krijgen voor deze cursus moet je aan de volgende voorwaarden voldoen:

Team:

- Je moet een projectmethodiek (scrum) toepassen;
- Je levert een werkende aangepaste applicatie op;
- Je levert een werkende CI/CD pipeline op;
- Je levert alle documentatie op (onderzoeksrapport, technischrapport + swagger, testrapport).

Individueel:

- Je voldoet aan de aanwezigheidseisen (zie einde paragraaf voor meer details);
- Je levert een significante bijdrage aan het proces;
- Je toont een betrokken/actieve houding binnen de cursus;
- Je bent een teamspeler;
- Je houdt je teamleden op de hoogte en houdt je aan de afspraken;
- Je levert een git-log aan van je contributie;
- Je levert een significante technische bijdrage aan het product (code, eisen, testen, etcetera);

Scenario's

Voldoe je tijdens de looptijd van de cursus niet aan één van bovenstaande criteria, dan ontvang je een officiële waarschuwing. Je krijgt dan de gelegenheid om de nodige wijzigingen aan te brengen om aan de ontbrekende criteria te voldoen, tenzij je niet aan de aanwezigheidseisen voldoet (niet herstelbaar).

- Als je slaagt binnen de eerste oplevering van de cursus, krijg je een eindcijfer;
- Lukt het niet om dit op tijd op te lossen, dan ontvang je bij de eerste poging een No Go/ND en zul je jezelf moeten bewijzen bij de herkansing;
- Als je er niet in slaagt de noodzakelijke wijzigingen aan te brengen en het niet mogelijk is om het probleem op te lossen, kun je de cursus niet opnieuw uitvoeren. Dit betekent dat je de cursus volgend studiejaar opnieuw moet volgen. De technisch docent en tutor beslissen of je bijdrage herstelbaar is of niet.

Aanwezigheid

Vanwege het belang van samenwerking is aanwezigheid bij de lessen verplicht. Je mag maximaal 4x keer afwezig zijn. Dit omvat ook afwezigheid om geldige redenen, zoals doktersafspraken, ziekte, rijexamens, etc. Te laat komen (binnenkomst na de roosterstarttijd) telt als afwezig. De aanwezigheid wordt tijdens elke les gecontroleerd door de docent(en).

Onder een les wordt verstaan: een gepland contactmoment volgens het rooster.

Voldoe je niet aan de aanwezigheidsplicht, dan ontvang je het resultaat ND (Not Done / Niet deelgenomen), tenzij anders geadviseerd door een decaan, en de docent daarmee akkoord gaat. Ontvang je een ND, dan kan de cursus niet binnen hetzelfde semester worden afgerond en moet deze opnieuw worden gevolgd in het eerstvolgende semester waarin de cursus wordt aangeboden.



3.2 Toetsmatrijs

	Leerdoelen	Niveau (Bloom)	Weging
L1	De student kan bestaande software analyseren, documenteren en valideren;	Analyseren, Onthouden	15%
L2	De student kan bestaande software verbeteren door refactoring technieken toe te passen om de code te verbeteren en uit te bereiden	Analyseren, Begrijpen	15%
L3	De student kan samenwerken met Git als versiebeheersysteem. Er kunnen branches gemaakt, samengevoegd en beheerd worden;	Toepassen	10%
L4	De student kan automatische tests schrijven en uitvoeren, waaronder unit tests en integratie tests	Toepassen	15%
L5	De student kan een CI/CD pipeline configureren binnen GitHub Actions of GitLab CI/CD met stappen voor build, testen en implementatie;	Analyseren, Toepassen	15%
L6	De student kent de basisprincipes van containerization en is in staat deze toe te passen met behulp van Docker of Podman	Toepassen	10%
L7	De student is zich bewust van beveiligingsaspecten, zoals het veilig opslaan van credentials, encryptie en het controleren van kwetsbaarheden zoals dependencies;	Evalueren	10%
L8	De student kan effectief samenwerken in een teamomgeving, waarbij code wordt beoordeeld, feedback wordt gegeven en communicatie plaatsvindt.	Creëren	10%



3.3 Rubric

Elk leerdoel wordt aan de hand van een rubric nagekeken (zie bijlage beoordelingsformulier). De rubric bestaat uit 4 onderdelen (onvoldoende, voldoende, goed en uitstekend), met daaronder een mogelijk puntenaantal behorende bij die rubric (zie tabel – Voorbeeld rubric).

Tabel - Voorbeeld rubric

	Onvoldoende					Voldoende		Goed		Uitstekend	
Omschrijving	<i>Niet alle verplichte onderdelen zijn uitgewerkt of de uitwerking is van onvoldoende kwaliteit.</i>					<i>Alle verplichte onderdelen zijn uitgewerkt, maar er is ruimte voor verbetering.</i>		<i>Alle verplichte onderdelen zijn goed uitgewerkt en extra onderdelen zijn deels toegepast.</i>		<i>Alle onderdelen zijn uitstekend uitgewerkt waarbij er extra onderdelen en/of best-practices zijn toegepast.</i>	
	1	2	3	4	5	6	7	8	9	10	
Argumentatie											



3.4 Becijfering

De toetsing vindt plaats op basis van het opgeleverde product (pipeline, code, documents) en het proces van het project (project-log, git-logs, contributieverlag). Alle leeruitkomsten moeten minimaal een voldoende scoren om in aanmerking te komen voor een voldoende voor de hele cursus. Individuele contributie kan een ND opleveren (zie 3.4) of een individuele aanpassing op het groeps cijfer tussen de -1 en +1 punt op basis van individuele contributie binnen de groep. Bij slechte prestatie kan er tot 1 punt van het groeps cijfer afgetrokken worden, bij uitstekende prestaties kan er tot 1 punt boven het groeps cijfer opgeteld worden (maximaal cijfer is 10).

Het eindcijfer wordt door zowel je tutor als je technisch docent bepaald. Voor elk leerdoel kan je maximaal 10 punten scoren. De volgende formule zal gebruikt worden voor de berekening van je eindscore:

$$\text{Eindcijfer} = \min\left(\sum_{n=1}^N \text{punten_leerdoel}_n \cdot \text{weging_leerdoel}_n + \text{individueel}, 10\right)$$

3.5 Cesuur

- Als je een **NVL** (*niet voldaan*) hebt gekregen voor aanwezigheid, dan is herkansing dit schooljaar niet meer mogelijk.
- Als je een **NVL** (*niet voldaan*) hebt gekregen voor contributie en reparatie is niet meer mogelijk, dan is herkansing dit schooljaar niet meer mogelijk.
- Als je een **NVL** (*niet voldaan*) hebt gekregen voor contributie en reparatie is mogelijk, dan kan je gebruikmaken van de herkansing (zie 3.5).
- Als je **eindcijfer en/of één van de leerdoelen** niet toereikend is (lager dan 5,5), dan kan je gebruikmaken van de herkansing (zie 3.5).

Voorbeeld:

Je krijgt het volgende cijfer:

- **(Individueel/team) Voldaan** voor projectmethodiek, aanwezigheid en bijdrage aan het proces, de pipeline en het product (vereiste requirements)
- Uit de rubriek een eindcijfer = **6,7 (voldoende)**

Je krijgt het volgende cijfer:

- **(Individueel/team) Voldaan** voor projectmethodiek, aanwezigheid en bijdrage aan het proces, de pipeline en het product (vereiste requirements)
- Uit de rubriek een eindcijfer = **5,4 (onvoldoende)**, je moet de herkansing doen.

Je krijgt het volgende cijfer:

- **(Individueel) Niet voldaan** voor individuele bijdrage (niet te repareren), **ND (niet deelgenomen)**, je komt niet in aanmerking voor de herkansing.

Je krijgt het volgende cijfer:

- **(Individueel) Niet voldaan** voor aanwezigheid, **ND (niet deelgenomen)**, je komt niet in aanmerking voor de herkansing.

3.6 Herkansing/ reparatie

Als het eindcijfer lager dan 5,5 is voor één of meer van de leerdoelen, of je hebt niet voldaan aan de minimumeisen, dan zal je gebruik moeten maken van de herkansing. Maximaal 2 leerdoelen mogen herkanst/gerepareerd worden, als er meer dan 2 leerdoelen onvoldoende zijn dan is herkansing niet mogelijk. Herkansing heeft een duur van 2 weken en dient voor het einde van week 20 ingeleverd te zijn.

Indien je de herkansing/reparatie niet haalt (lager dan 5,5) of als je een ND krijgt dan zal je deze cursus volgend schooljaar opnieuw moeten doen.



4 Plagiaat en richtlijnen voor het gebruik van GenAI

Schrijf zelf je code en documentatie

Het gebruik van enige vorm van GenAI voor het genereren van code of documentatie is niet toegestaan. Bij de beoordeling van leerdoelen wordt het begrip en de toepassing van de concepten door het docententeam geëvalueerd. Het indienen van AI-gegenereerde code of documentatie ondermijnt dit proces en maakt de leerdoelen ongeldig. Als de inzending AI-gegenereerde inhoud bevat, riskeer je een ND.

Ethisch gebruik

Het gebruik van GenAI is alleen toegestaan als ondersteunend hulpmiddel in je leerproces. Denk bijvoorbeeld aan het uitleggen van abstracte theoretische programmeerconcepten, het genereren van grote hoeveelheden seed-data, of het toelichten van specifieke compiler-foutmeldingen uit je eigen code. GenAI is een hulpmiddel en geen vervanging voor zelfstandig nadenken. Alleen door zelf stapsgewijs problemen op te lossen bouw je ervaring op. Je bent zelf verantwoordelijk om na te blijven denken bij de inzet van GenAI.

Controleer informatie

Door GenAI gegenereerde inhoud moet altijd worden gecontroleerd en geverifieerd aan de hand van betrouwbare (wetenschappelijke) bronnen. AI-toepassingen kunnen fouten bevatten of verouderde informatie genereren.

Voorkom plagiaat

Plagiaat in welke vorm dan ook wordt niet getolereerd en resulteert eveneens in een ND. Werk dat niet door de student zelf is geschreven wordt beschouwd als plagiaat. Je dient kritisch na te denken, teksten te herschrijven en altijd correct te verwijzen. Gebruik voor bronverwijzingen APA- of IEEE-formuleringen. Verwijs naar GenAI-bronnen volgens de APA7 richtlijn: <https://apastyle.apa.org/blog/cite-generative-ai-references>.

Transparantie

Wanneer je GenAI gebruikt (wat alleen toegestaan is zoals omschreven bij 'Ethisch gebruik', en niet om je code of documentatie te schrijven), vermeld dan expliciet op welke manier je GenAI hebt gebruikt. Bewaar chatlogs en toon deze aan de docent wanneer hierom wordt gevraagd. Hiermee ben je transparant over je werkwijze.



Bijlage I - Uitleg op te leveren producten

Presentaties

Presentatie 1:

Binnen deze presentatie neem je ons mee in de bevindingen welke jullie als groep hebben opgedaan na het analyseren van de bestaande software. Je stipt de belangrijkste kenmerken aan en beschrijft wat functionele en niet functionele requirements.

Presentatie 2:

Binnen deze presentatie neem je ons mee in de tests welke jullie als groep hebben opgesteld voor de bestaande applicatie. Jullie vertellen over de uit te voeren testsoorten die jullie gaan implementeren en waarom en welke tests daar o.a. onder vallen.

Presentatie 3:

In deze eindpresentatie nemen jullie als groep ons mee door het project. Je vertelt kort over jullie analyse, de requirements die daaruit kwamen en de validatie ervan. Welke tests er opgezet zijn en waarom deze gekozen zijn. Hoe de refactoring heeft plaatsgevonden en wat het resultaat ervan is en eventueel welke uitbreidingen er gemaakt zijn. Ook laat je de CI/CD pipeline zien en welke verbeteringen op documentatieniveau er gemaakt zijn om de uitbreidbaarheid van de software te vergroten.

Rapporten

Voor alle rapporten geldt dat ze in het Engels of het Nederlands geschreven dienen te worden waarbij het gehele rapport geschreven is in dezelfde taal. De keuze voor de taal is aan de student tenzij de docent anders aangeeft.

Onderzoeksrapport:

Het onderzoeksrapport geeft een duidelijk inzicht in hoe jullie als groep te werk zijn gegaan bij het analyseren van de bestaande software en welke requirements daaruit onttrokken zijn. Hiernaast geven jullie aan hoe jullie deze requirements hebben gevalideerd en welke resultaten dit opgeleverd heeft. Tot slot sluit je af met de conclusie en eventuele aanbevelingen. Dit alles verwerk je netjes in een rapportage.

Testrapport:

Het testrapport geeft een duidelijk inzicht in hoe jullie als groep de software getest hebben, welke methodes en technieken ervoor gebruikt zijn en wat de uitkomsten hiervan waren.

Technisch rapport:

Het technisch rapport geeft een gedetailleerde weergave van het technische onderdeel van de applicatie. Doormiddel van het technisch rapport zou een ander in staat moeten zijn te starten en/of verder te gaan met de ontwikkeling/uitbreiding van de applicatie.

**Koppelmatrix**

In onderstaande tabel is terug te vinden welke producten gekoppeld zijn aan welke leerdoelen.

Product(en)	Leerdoelen
Projectdocumentatie (scrum)	L3, L8
Werkende CI/CD pipeline	L5, L6, L7
Aangepaste applicatie	L2, L3, L4, L7
Onderzoeksrapport	L1, L6, L7
Technisch rapport	L2, L3, L6, L7
Testrapport	L2, L4, L5
Swagger documentatie	L1, L2
Presentaties	L1, L2, L4, L5, L6, L7
Bewijs individuele bijdrage	L1, L2, L3, L4, L5, L6, L7, L8



Bijlage II - Beoordelingsformulier

L1	De student kan bestaande software analyseren, documenteren en verifiëren									
	Onvoldoende					Voldoende		Goed		Uitstekend
Omschrijving	<i>De student heeft de software niet geanalyseerd, gedocumenteerd of geverifieerd.</i> <i>Er zijn ernstige tekortkomingen in de uitvoering van de opdracht.</i>					<i>De student heeft de software gedeeltelijk geanalyseerd, gedocumenteerd en geverifieerd.</i> <i>Er zijn requirements opgesteld en heeft enige vorm van verificatie plaatsgevonden.</i> <i>De belangrijkste tekortkomingen in de applicatie zijn gedetecteerd en er is een voorstel voor verbetering gedaan.</i> <i>De API bevat bruikbare Swagger documentatie.</i>		<i>De student heeft de software volledig geanalyseerd, gedocumenteerd en geverifieerd.</i> <i>Requirements zijn meetbaar, eenduidig en compleet.</i> <i>Het verbetervoorstel is goed onderbouwd met betrouwbare bronnen.</i> <i>De documentatie is zo opgesteld dat deze bruikbaar is voor iemand met minimale kennis van het product.</i>		<i>De student heeft de software uitmuntend geanalyseerd, gedocumenteerd en geverifieerd.</i> <i>Bij het opstellen van de requirements is rekening gehouden met de context, en dit is goed gevalideerd.</i> <i>In het verbetervoorstel is een wel overwogen afweging gemaakt tussen verschillende opties, en hiervoor is een onderbouwde keuze gemaakt.</i> <i>De documentatie is compleet en stelt een nieuw team in staat om zonder grote problemen aan het project verder te werken.</i>
	1	2	3	4	5	6	7	8	9	10
Argumentatie										



L2	De student kan effectieve refactoring technieken toe passen om de code te verbeteren en uit te bereiden									
	Onvoldoende					Voldoende		Goed		Uitstekend
Omschrijving	<i>Er zijn geen refactoring technieken toegepast om de code te verbeteren.</i> <i>Er is veel duplicate code of de code is niet logisch opgedeeld.</i> <i>Er zijn geen uitbereidingen uitgevoerd.</i>					<i>De student heeft de bestaande code gedeeltelijk verbeterd.</i> <i>Er is weinig duplicate code en de code is logisch opgedeeld. De code bevat relevante comments.</i> <i>Er zijn enkele uitbereidingen uitgevoerd</i>		<i>De student heeft de bestaande code significant verbeterd.</i> <i>Er zijn design patterns toegepast om coupling en dependencies te beperken met het oog op maintainability.</i> <i>Alle belangrijke verbeteringen zijn doorgevoerd</i>		<i>De student heeft de bestaande code grondig geanalyseerd en alle relevante verbeterpunten geïdentificeerd en uitgevoerd..</i> <i>De code is op een dusdanig logische manier opgezet dat een nieuw team zonder grote problemen verder kan werken aan het project, en dit makkelijk kan uitbereiden.</i> <i>Uitbereidingen zijn goed geverifieerd met de PO en gaan voorbij aan de opdracht.</i>
	1	2	3	4	5	6	7	8	9	10
Argumentatie										



L3	De student kan samenwerken met Git als versiebeheersysteem. Er kunnen branches gemaakt, samengevoegd en beheerd worden									
	Onvoldoende					Voldoende		Goed		Uitstekend
Omschrijving	<i>De student heeft geen ervaring met Git aangetoond.</i> <i>Er zijn geen branches aangemaakt, samengevoegd of beheert.</i> <i>Pullrequests worden niet gebruikt.</i>					<i>De student heeft basiskennis van Git aangetoond.</i> <i>Er zijn eenvoudige branches aangemaakt en samengevoegd en branches worden beheerd met minimale fouten.</i> <i>Er wordt enige vorm van feedback gegeven op pullrequests.</i>		<i>De student heeft een goede kennis van Git aangetoond.</i> <i>Er zijn complexere branches aangemaakt en samengevoegd en branches worden efficiënt beheert met begrip van Git-flows.</i> <i>Er worden duidelijke verbeterpunten geïdentificeerd met behulp van pullrequests.</i>		<i>De student heeft diepgaande kennis van Git aangetoond.</i> <i>Er zijn geavanceerde Git-technieken toegepast en branches worden foutloos beheert en geoptimaliseerd via workflows.</i> <i>Pullrequests worden goed toegepast om zowel de functionaliteit als de onderhoudbaarheid van de code te verbeteren.</i>
	1	2	3	4	5	6	7	8	9	10
Argumentatie										



L4	De student kan automatische tests schrijven en uitvoeren, waaronder unit tests en integratie tests									
	Onvoldoende					Voldoende		Goed		Uitstekend
Omschrijving	De student heeft geen ervaring met automatische tests. Er zijn geen unit tests of integratietests geschreven.					De student heeft basiskennis van automatische tests. Er zijn eenvoudige unit en integratie tests geschreven en uitgevoerd waarmee de essentiële onderdelen van de code gedekt is. Er is enig begrip van het belang van testdekking.		De student heeft een goede kennis van automatische tests. Er zijn complexere unit tests en integratietests geschreven en uitgevoerd waarbij rekening is gehouden met de belangrijkste randgevallen. Er is begrip van het testproces, waarbij testresultaten geanalyseerd worden voor verbeteringen.		De student heeft diepgaande kennis van automatische tests. Er zijn geavanceerde unit tests en integratietests geschreven en uitgevoerd en teststrategieën zijn geoptimaliseerd aan de hand van best-practices. Er zijn tests uitgevoerd die voorbijgaan aan standaard unit/integratie tests.
	1	2	3	4	5	6	7	8	9	10
Argumentatie										



L5	De student kan een CI/CD pipeline configureren binnen GitHub Actions of GitLab CI/CD met stappen voor build, testen en deployment									
	Onvoldoende					Voldoende		Goed		Uitstekend
Omschrijving	<i>De student heeft geen ervaring met CI/CD-pipelines.</i> <i>Er is geen basisconfiguratie voor GitHub Actions of GitLab CI/CD opgezet.</i>					<i>De student heeft basiskennis van CI/CD-pipelines.</i> <i>Er is een eenvoudige pipeline geconfigureerd met stappen voor build en testen.</i>		<i>De student heeft een goede kennis van CI/CD-pipelines.</i> <i>Er is een complexere pipeline geconfigureerd met stappen voor build, test en deploy.</i>		<i>De student heeft diepgaande kennis van CI/CD.</i> <i>Er is een geavanceerde pipeline geconfigureerd met integratie van externe services. En de pipeline is geoptimaliseerd voor snelheid, betrouwbaarheid en veiligheid.</i>
	1	2	3	4	5	6	7	8	9	10
Argumentatie										



L6 De student kent de basis principes van containerization en is in staat deze toe te passen met behulp van Docker of Podman										
	Onvoldoende					Voldoende		Goed		Uitstekend
Omschrijving	<i>De student heeft geen begrip van containerization.</i> <i>Er wordt geen gebruik gemaakt van containers, of de container is niet functioneel.</i>					<i>De student heeft een basisbegrip van containerization.</i> <i>Er is een functionele container waarbinnen de code draait die gebruikt kan worden voor ontwikkel en/of testing doeleinden.</i>		<i>De student heeft een goede kennis containerization.</i> <i>Er is een functionele container die bruikbaar is in de productie omgeving. Data wordt persistent opgeslagen.</i>		<i>De student heeft diepgaande kennis van containerization.</i> <i>Er wordt gebruik gemaakt van meerdere containers die op een geavanceerde manier samen werken. De productie omgeving is opgezet voor zero downtime deployments.</i>
	1	2	3	4	5	6	7	8	9	10
Argumentatie										



L7	De student is zich bewust van beveiligingsaspecten, zoals het veilig opslaan van credentials, encryptie en het controleren van kwetsbaarheden zoals dependencies									
	Onvoldoende					Voldoende		Goed		Uitstekend
Omschrijving	<i>De student heeft geen begrip van beveiligingsaspecten.</i> <i>Credentials zijn niet veilig opgeslagen, er is geen encryptie toegepast en controle van kwetsbaarheden zoals dependencies is niet uitgevoerd.</i>					<i>De student heeft een basisbegrip van beveiligingsprincipes.</i> <i>Credentials zijn veilig opgeslagen, er wordt gebruik gemaakt van encryptie bij opslag van personsgegevens.</i>		<i>De student heeft diepgaand begrip van hoe beveiliging werkt.</i> <i>Er worden effectieve maatregelen genomen tegen mogelijke kwetsbaarheden in dependencies en de eigen code.</i>		<i>De student heeft een uitgebreid begrip van beveiligingspraktijken.</i> <i>Beveiligingsmaatregelen worden proactief geïmplementeerd en best practices worden gevolgd.</i>
	1	2	3	4	5	6	7	8	9	10
Argumentatie										



L8 De student kan effectief samenwerken in een teamomgeving, waarbij code wordt beoordeeld, feedback wordt gegeven en communicatie plaatsvindt.										
Omschrijving	Onvoldoende					Voldoende		Goed		Uitstekend
	<i>De student communiceert beperkt met teamleden.</i> <i>Deelt informatie niet of te laat, en geeft of verwerkt nauwelijks feedback.</i> <i>Scrum rituelen zijn niet of nauwelijks uitgevoerd.</i>					<i>De student communiceert tijdig over eigen werkzaamheden.</i> <i>Geeft en verwerkt eenvoudige feedback.</i> <i>Scrumrituelen zijn overwegend volgens de afspraken uitgevoerd.</i>		<i>De student communiceert duidelijk en respectvol.</i> <i>Geeft onderbouwde feedback en verwerkt ontvangen feedback zichtbaar.</i> <i>Scrumrituelen zijn effectief uitgevoerd waarbij verbeterpunten zijn geëvalueerd en toegepast.</i>		<i>De student communiceert proactief en bevordert de samenwerking binnen het team.</i> <i>Scrumrituelen zijn effectief uitgevoerd waarbi zowel het ontwikkelproces als de samenwerking significant verbeterd zijn.</i>
Argumentatie	1	2	3	4	5	6	7	8	9	10



Software Construction: Processing and Tools

Klas		Team		Poging	1 / 2	
Leerdoelen					Weging	Punten
L1	De student kan bestaande software analyseren, documenteren en valideren				15%	
L2	De student kan effectieve refactoring technieken toe passen om de code te verbeteren en uit te bereiden				15%	
L3	De student kan samenwerken met Git als versiebeheersysteem. Er kunnen branches gemaakt, samengevoegd en beheerd worden				10%	
L4	De student kan automatische tests schrijven en uitvoeren, waaronder unit tests en integratie tests				15%	
L5	De student kan een CI/CD pipeline configureren binnen GitHub Actions of GitLab CI/CD met stappen voor build, testen en implementatie				15%	
L6	De student kent de basisprincipes van containerization en is in staat deze toe te passen met behulp van Docker of Podman				10%	
L7	De student is zich bewust van beveiligingsaspecten, zoals het veilig opslaan van credentials, encryptie en het controleren van kwetsbaarheden dependencies				10%	
L8	De student kan effectief samenwerken in een teamomgeving, waarbij code wordt beoordeeld, feedback wordt gegeven en communicatie plaatsvindt				10%	
Vereisten						
Studentnaam	{naam}	{naam}	{naam}	{naam}	{naam}	
Studentnummer	{nummer}	{nummer}	{nummer}	{nummer}	{nummer}	
Aanwezigheid	VLD / NVL	VLD / NVL	VLD / NVL	VLD / NVL	VLD / NVL	
Contributie aan het proces	VLD / NVL	VLD / NVL	VLD / NVL	VLD / NVL	VLD / NVL	
Contributie aan het product	VLD / NVL	VLD / NVL	VLD / NVL	VLD / NVL	VLD / NVL	
Contributie aan de pipeline	VLD / NVL	VLD / NVL	VLD / NVL	VLD / NVL	VLD / NVL	
Correctie voor inspanning	-1 / 0 / +1	-1 / 0 / +1	-1 / 0 / +1	-1 / 0 / +1	-1 / 0 / +1	
Resultaat						
Eindcijfer						

Datum:

Technisch docent

Naam:

Handtekening:

Tutor

Naam:

Handtekening:



Extra onderbouwing:

Omschrijving reparatie: